



Université Joseph KI-
ZERBO



UFR SEA

RAPPORT DE CONCEPTION DE MOTEUR DE RECHERCHE SCIENTIFIQUE

<i>NOM</i>	<i>PRENOM</i>	<i>MAIL</i>
BARRO	Mamadou	barromamadou6389@gmail.com

23 mai 2025

Table des matières

Introduction	1
Problématique	1
Objectifs	2
État de l’art	2
Conception du système	3
Technique d’indexation utilisée dans le projet	4
Réalisation du système	5
Résultats et tests	8
Conclusion et perspectives	9

Table des figures

1	interface utilisateur	8
---	---------------------------------	---

Introduction

La croissance exponentielle des publications scientifiques rend de plus en plus difficile l'accès rapide et pertinent à l'information pour les chercheurs, enseignants, étudiants et professionnels du savoir. Chaque jour, des milliers d'articles sont mis en ligne, couvrant des domaines variés et souvent publiés dans différentes langues et formats. Dans ce contexte, les outils de recherche classiques, bien qu'efficaces pour des usages généralistes, ne répondent pas toujours aux besoins spécifiques de la communauté scientifique.

Les moteurs de recherche généralistes comme Google ou Bing n'intègrent pas nécessairement des critères pertinents pour la recherche académique : filtrage par langue, classement par pertinence scientifique, accès au texte intégral, etc. Quant aux moteurs spécialisés tels que Google Scholar, ResearchGate ou Semantic Scholar, bien qu'utiles, ils présentent des limites non négligeables : difficulté d'accès aux documents en texte intégral, manque de transparence sur les algorithmes de classement, interfaces parfois restreintes, ou encore absence de filtrage avancé sur certains critères comme la langue ou le type de document.

Dans cette optique, le présent projet s'inscrit dans une volonté de proposer une solution simple, personnalisable et locale pour faciliter l'accès à l'information scientifique en français. Il s'agit de concevoir et de développer un moteur de recherche de documents scientifiques (notamment des fichiers PDF), qui intègre un processus d'indexation, de recherche par mots-clés, de filtrage linguistique et d'affichage des résultats enrichis. Le tout repose sur une infrastructure légère, adaptée à un usage académique ou pédagogique.

Ce rapport présente de manière détaillée les différentes étapes de réalisation du projet, depuis la conception jusqu'à la mise en œuvre technique, en passant par les choix méthodologiques, les technologies utilisées et les perspectives d'amélioration.

Problématique

La recherche scientifique joue un rôle fondamental dans le développement des connaissances, la résolution des problèmes sociétaux et l'innovation technologique. Toutefois, l'explosion du volume de données disponibles sur Internet rend complexe l'identification rapide et pertinente des documents scientifiques utiles à une recherche donnée.

Les moteurs de recherche généralistes, tels que Google, fonctionnent sur des critères de référencement global, souvent influencés par la popularité des pages, la publicité ou des optimisations commerciales. Ces outils, bien qu'efficaces pour des recherches larges, manquent de précision lorsqu'il s'agit de retrouver des articles académiques spécifiques, en particulier lorsqu'ils sont rédigés dans des langues autres que l'anglais, comme le français.

Par ailleurs, les moteurs de recherche spécialisés comme Google Scholar, HAL ou ResearchGate, bien qu'ils offrent un accès plus ciblé à la littérature scientifique, présentent aussi des limites : accès partiel aux textes intégraux, résultats en langue anglaise majoritaires, manque de transparence sur les critères de tri, ou encore difficulté d'adaptation à des contextes pédagogiques locaux.

Dès lors, il apparaît nécessaire de concevoir un outil léger, local et centré sur la recherche de documents scientifiques en français. Cet outil doit permettre non seulement une indexation efficace des documents disponibles (notamment au format PDF), mais aussi une recherche pertinente avec

surlignage des mots-clés, filtrage par langue, et génération de statistiques pour évaluer l'usage du moteur.

Objectifs

L'objectif principal de ce projet est de concevoir et de développer un moteur de recherche local permettant l'accès rapide et pertinent à des documents scientifiques, en particulier ceux rédigés en langue française.

Ce moteur vise à offrir une solution personnalisée, légère et fonctionnelle, adaptée aux contextes académiques, notamment dans les environnements où l'accès à Internet est limité ou les ressources spécialisées sont peu accessibles.

Objectifs spécifiques

- Mettre en place une architecture logicielle permettant l'indexation de documents scientifiques au format PDF.
- Développer une interface web simple et intuitive pour la recherche par mots-clés.
- Intégrer un mécanisme de filtrage linguistique, en se concentrant sur les documents rédigés en français.
- Mettre en place une fonctionnalité de surlignage des mots-clés dans les résultats affichés.
- Générer des statistiques d'usage du moteur de recherche (nombre de requêtes, mots-clés les plus utilisés, documents les plus consultés, etc.).
- Garantir une utilisation hors-ligne, sécurisée et adaptée à des bases documentaires locales.

État de l'art

Le domaine des moteurs de recherche a connu un essor considérable depuis les années 1990. Les outils généralistes comme *Google*, *Bing* ou *Yahoo* dominent le marché et permettent aux utilisateurs de retrouver rapidement une information parmi des milliards de pages web. Ces moteurs reposent sur des algorithmes sophistiqués de classement, souvent basés sur la popularité des pages, la structure des liens (PageRank), ou encore des techniques d'apprentissage automatique.

Moteurs de recherche généralistes

Les moteurs généralistes offrent des résultats rapides mais souvent peu adaptés à des recherches académiques. Par exemple, une recherche portant sur « réseaux de neurones convolutifs » peut retourner des blogs, vidéos, ou pages promotionnelles avant de fournir des articles scientifiques pertinents. De plus, ces moteurs privilégient souvent les contenus en anglais, ce qui constitue une barrière pour une partie du public francophone.

Moteurs de recherche scientifiques

Les moteurs spécialisés comme **Google Scholar**, **Semantic Scholar**, **BASE**, ou encore **HAL** sont mieux adaptés au contexte académique. Ils permettent d'accéder à des publications scientifiques, des thèses, des rapports, et des revues.

Cependant, plusieurs limites subsistent :

- L'accès aux textes intégraux est souvent restreint ou payant.
- La recherche en langue française y est minoritaire.
- Les algorithmes de classement ne sont pas personnalisables.
- L'utilisation hors ligne n'est pas envisageable.

Moteurs personnalisés et locaux

Face à ces limites, plusieurs chercheurs et institutions ont proposé des moteurs locaux basés sur des outils open source comme **Apache Lucene**, **ElasticSearch** ou **Whoosh**. Ces moteurs offrent une grande flexibilité et permettent de mettre en place des solutions sur mesure pour l'indexation et la recherche locale de documents (par exemple, des PDF hébergés dans un répertoire ou une base interne).

Le présent projet s'inscrit dans cette logique, en exploitant **Whoosh** et **PyPDF2** pour mettre en place un moteur simple, local, en français, avec un surlignage des mots-clés et une interface web légère basée sur **Flask**.

Conception du système

La phase de conception vise à définir l'architecture générale du moteur de recherche, les modules fonctionnels, et les interactions entre les composants. Le moteur proposé est conçu selon une architecture en couches afin de faciliter son extensibilité, sa maintenabilité et sa clarté.

Architecture générale

Le système repose sur quatre composants principaux :

1. **Module d'indexation** : extrait le texte des documents PDF (à l'aide de `PyPDF2`) et alimente un index inversé à l'aide de la bibliothèque `Whoosh`.
2. **Module de recherche** : traite les requêtes des utilisateurs, interroge l'index, filtre les résultats par langue (français uniquement), et applique le surlignage des mots-clés.
3. **Interface web** : développée avec `Flask`, elle permet aux utilisateurs d'effectuer leurs recherches, d'afficher les résultats avec surlignage, et d'accéder aux statistiques de recherche.
4. **Module de statistiques** : enregistre les requêtes de recherche et génère des rapports simples sur l'utilisation (fréquence des mots-clés, nombre de recherches, etc.).

Technique d'indexation utilisée dans le projet

Dans le cadre de ce projet, nous avons mis en œuvre une **indexation inversée statique**, à l'aide de la bibliothèque `Whoosh`, spécialisée pour la recherche textuelle en Python.

Méthode utilisée : Index inversé (non naïve) Contrairement à la méthode dite *naïve*, qui consiste à parcourir à chaque requête tous les documents ligne par ligne, nous avons utilisé une structure optimisée : l'**index inversé**. Cette structure associe chaque mot du vocabulaire à la liste des documents dans lesquels il apparaît.

- Type : Index inversé statique
- Outil : Bibliothèque Whoosh
- Contenu indexé : `title`, `content`, `filename`
- Sources : Fichiers PDF extraits localement avec `PyPDF2`
- Filtrage : uniquement les documents rédigés en français (`langdetect`)

Formule de pondération (modèle vectoriel) Whoosh implémente un **modèle vectoriel** classique, basé sur la pondération TF-IDF, même si cela est géré de manière interne et transparente pour le développeur. La pondération d'un terme t_i dans un document d est donnée par :

$$w_{i,d} = tf_{i,d} \cdot idf_i$$

avec :

- $tf_{i,d}$: fréquence du terme t_i dans le document d
- $idf_i = \log\left(\frac{N}{df_i}\right)$, où N est le nombre total de documents et df_i le nombre de documents contenant t_i

Algorithme d'indexation utilisé L'algorithme d'indexation développé dans le script `indexer.py` fonctionne comme suit :

1. Parcours de tous les fichiers PDF dans le dossier `corpus/`
2. Extraction du texte à l'aide de `PyPDF2`
3. Détection de la langue (on ne garde que les fichiers en français)
4. Création d'un enregistrement dans l'index avec les champs :
 - `title` : nom du fichier
 - `content` : texte intégral extrait
 - `filename` : nom du fichier pour l'accès utilisateur
5. Construction automatique de l'index inversé par Whoosh

Exemple de structure indexée

```
{
  "title": "web_semantic.pdf",
  "content": "The Semantic Web is an extension of the current web...",
  "filename": "web_semantic.pdf"
}
```

Conclusion L'utilisation de Whoosh permet de bénéficier d'une indexation efficace, d'une gestion intégrée du modèle vectoriel avec TF-IDF, et d'une recherche rapide via un index inversé. Cette approche garantit un bon compromis entre performance, simplicité et pertinence des résultats.

Modèle de données

L'index inversé contient pour chaque document les champs suivants :

- **title** : le nom du fichier PDF.
- **content** : le contenu textuel extrait.
- **path** : le chemin d'accès au fichier.
- **lang** : la langue détectée automatiquement (via `langdetect`).

Contraintes

- L'application doit fonctionner en local sans dépendre d'un service tiers.
- Seuls les documents en langue française doivent être indexés et recherchés.
- L'interface doit être simple, responsive et rapide.

Réalisation du système

La réalisation du projet a été faite en langage **Python**, avec le framework web **Flask** pour l'interface utilisateur. L'indexation repose sur la bibliothèque **Whoosh**, tandis que l'extraction de texte des fichiers PDF est assurée par **PyPDF2**. Pour la détection de langue, nous utilisons `langdetect`.

Structure du projet

```
moteur_scientifique/  
  app.py  
  indexer.py  
  stats.py  
  templates/  
    index.html  
  static/  
    style.css  
  pdfs/  
    [documents PDF]  
  indexdir/  
  recherches.json
```


1. Indexation des documents

Le script `indexer.py` permet d'indexer tous les documents PDF du dossier `pdfs/`, uniquement s'ils sont rédigés en français. Voici un extrait du code principal :

```
from whoosh.fields import Schema, TEXT, ID
from whoosh.index import create_in
from langdetect import detect
from PyPDF2 import PdfReader
import os

schema = Schema(title=TEXT(stored=True),
                 content=TEXT(stored=True),
                 path=ID(stored=True),
                 lang=ID(stored=True))

if not os.path.exists("indexdir"):
    os.mkdir("indexdir")

ix = create_in("indexdir", schema)
writer = ix.writer()

for filename in os.listdir("pdfs"):
    if filename.endswith(".pdf"):
        reader = PdfReader(os.path.join("pdfs", filename))
        content = "".join([page.extract_text() or "" for page in reader.pages])
        try:
            if detect(content) == "fr":
                writer.add_document(title=filename, content=content,
                                   path=os.path.abspath(os.path.join("pdfs",
                                                                       filename)),
                                   lang="fr")
        except:
            continue

writer.commit()
```

2. Recherche et surlignage

La recherche est assurée par `app.py`, qui interroge l'index à partir d'une requête. Seuls les documents en français sont retournés, et les mots-clés sont surlignés :

```
from flask import Flask, render_template, request
from whoosh.index import open_dir
from whoosh.qparser import QueryParser
import json, os, re
```

```

app = Flask(__name__)

@app.route("/", methods=["GET", "POST"])
def index():
    results, query = [], ""
    if request.method == "POST":
        query = request.form["query"]
        ix = open_dir("indexdir")
        with ix.searcher() as searcher:
            parser = QueryParser("content", schema=ix.schema)
            myquery = parser.parse(query)
            hits = searcher.search(myquery, limit=20)
            for hit in hits:
                if hit['lang'] == 'fr':
                    content = hit['content']
                    for kw in query.split():
                        content = re.sub(f"(?i)({kw})", r"<mark>\1</mark>",
                                         results.append({
                                             "title": hit['title'],
                                             "path": hit['path'],
                                             "snippet": content[:500] + "...")
                    })
            log_search(query)
    return render_template("index.html", results=results, query=query)

def log_search(q):
    data = []
    if os.path.exists("recherches.json"):
        with open("recherches.json", "r") as f:
            data = json.load(f)
    data.append(q)
    with open("recherches.json", "w") as f:
        json.dump(data, f)

```

3. Interface utilisateur

L'interface web `index.html` permet de saisir une requête et d'afficher les résultats surlignés. Elle est simple et responsive, adaptée à une utilisation académique.

```

<form method="post">
  <input type="text" name="query" placeholder="Rechercher un article..." re
  <button type="submit">Rechercher</button>
</form>

{% for r in results %}

```

```
<div class="result">
  <strong>{{ r.title }}</strong><br>
  <a href="{{ r.path }}">Ouvrir le document</a><br>
  <p>{{ r.snippet|safe }}</p>
</div>
{% endfor %}
```

4. Statistiques

Le fichier `recherches.json` enregistre chaque mot-clé recherché. Des statistiques peuvent être affichées (mots les plus fréquents, nombre de recherches) via un script ou une interface complémentaire.

Résultats et tests

1. Fonctionnement général

Après avoir indexé un ensemble de documents scientifiques au format PDF, le moteur de recherche est accessible via une interface web simple. L'utilisateur saisit un ou plusieurs mots-clés dans un champ de recherche, et les documents pertinents sont affichés en quelques secondes. Chaque résultat contient :

- Le titre du document,
- Un lien direct vers le fichier PDF local,
- Un extrait du contenu du document contenant le ou les mots-clés recherchés, avec surlignage visuel.



FIGURE 1 – interface utilisateur

4. Performances

Des tests ont été réalisés sur un ensemble de **3 fichiers PDF** scientifiques de tailles variées (entre 100 Ko et 2 Mo chacun). L'indexation a pris en moyenne **3 à 5 secondes** par document, selon la qualité de l'extraction du texte. Les requêtes de recherche s'exécutent en moins de **0.5 seconde**, y compris pour des expressions complexes.

5. Statistiques de recherche

Toutes les requêtes saisies par les utilisateurs sont enregistrées dans un fichier `recherches.json`. Un traitement ultérieur de ces données permet de générer des statistiques telles que :

- Les mots-clés les plus recherchés,
- Le nombre total de recherches effectuées,
- Les tendances de recherche sur une période donnée.

Ces statistiques peuvent être utiles pour analyser les besoins documentaires des utilisateurs, adapter l'index ou enrichir les corpus.

6. Limites observées

- L'extraction de texte dépend fortement de la qualité du fichier PDF. Certains documents scannés ou mal formatés ne permettent pas une indexation correcte.
- Le moteur ne traite pas encore les synonymes ou la recherche sémantique.
- L'absence de base de données empêche une gestion plus structurée des résultats ou des utilisateurs.

Conclusion et perspectives

Conclusion

Ce projet a permis la conception et la réalisation d'un moteur de recherche de documents scientifiques, ciblant spécifiquement les fichiers rédigés en français. Il s'attaque à une problématique réelle : l'accès rapide, pertinent et local à la connaissance scientifique dans un contexte académique.

Le moteur développé offre les fonctionnalités suivantes :

- Indexation automatique de fichiers PDF,
- Recherche par mot-clé avec affichage d'extraits pertinents,
- Surlignage des mots-clés dans les résultats,
- Filtrage des documents selon la langue (français),
- Enregistrement des requêtes pour statistiques.

Grâce à l'utilisation de bibliothèques telles que Whoosh, PyMuPDF et Flask, une solution simple mais fonctionnelle a pu être mise en place en local, avec une interface web intuitive.

Perspectives

Bien que les résultats soient prometteurs, plusieurs pistes d'amélioration sont identifiées :

- **Recherche avancée** : ajouter le support de la recherche booléenne (AND, OR, NOT), de la racinisation (stemming) ou de la recherche approximative (fuzzy search).
- **Amélioration de l'interface** : proposer une navigation plus fluide, un affichage paginé des résultats, ou un téléchargement direct.
- **Accès distant** : rendre le moteur accessible à distance via un serveur sécurisé pour une utilisation par une communauté d'utilisateurs (étudiants, enseignants...).
- **Base de données** : intégrer une base de données (par exemple SQLite ou PostgreSQL) pour une meilleure gestion des métadonnées, des utilisateurs et des statistiques.
- **Classification thématique** : regrouper les documents indexés par domaines scientifiques, à l'aide de méthodes de traitement automatique du langage naturel.

Ainsi, ce projet constitue une première étape vers une plateforme documentaire plus intelligente et personnalisée, au service de la recherche et de l'éducation francophone.